

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Задание № 3

По дисциплине

“Алгоритмы компьютерной графики”

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р3413

Преподаватель: Андреев Артем Станиславович

Санкт-Петербург, 2025г

Задание

Проверить, как задаются и передаются последовательность матриц.

Помножить точки на матрицы в вершинном шейдере получая последовательные промежуточные результаты, затем одной WorldViewProj матрицей.

Уметь визуализировать результат – координаты на поверхности треугольников, правильно передав промежуточные значения из VS в FS.

Ход работы

Получаем 3 варианта визуализации, каждый из которых находится в пиксельном шейдере.

- 1) Визуализация мировых координат как цвета
- 2) Визуализация видовых координат
- 3) Визуализация только положительных мировых координат

Исходный код:

```
// Объявляем матрицы
float4x4 World : World;
float4x4 View : View;
float4x4 Projection : Projection;
float4x4 WorldViewProj : WorldViewProjection;

// Структура ВХОДА вершинного шейдера (данные модели)
struct VS_INPUT {
    float3 Pos : POSITION;
    float3 Normal : NORMAL;
};

// Структура ВЫХОДА вершинного шейдера (то, что идет в пиксельный шейдер)
struct VS_OUTPUT {
    float4 PosH : POSITION; // Итоговая позиция на экране
    // Промежуточные значения для визуализации:
    float3 WorldPos : TEXCOORD1; // Позиция в мировом пространстве
    float3 ViewPos : TEXCOORD2; // Позиция в пространстве камеры
    float3 ObjectPos : TEXCOORD3; // Исходная позиция (опционально)
};

// Вершинный шейдер
VS_OUTPUT VS_MatrixTest(VS_INPUT input)
{
    VS_OUTPUT output;

    // 1. Исходная локальная позиция (просто для передачи)
    output.ObjectPos = input.Pos;

    // 2. Преобразуем в мировое пространство: Pos_World = Pos_Object * World
    float4 worldPos = mul(float4(input.Pos, 1.0), World);
    output.WorldPos = worldPos.xyz; // Сохраняем для передачи

    // 3. Преобразуем в пространство камеры: Pos_View = Pos_World * View
    float4 viewPos = mul(worldPos, View);
    output.ViewPos = viewPos.xyz; // Сохраняем для передачи

    // 4. Преобразуем в экранное пространство: Pos_Project = Pos_View *
    Projection
    float4 projPos = mul(viewPos, Projection);
    output.PosH = projPos;
```

```

    return output;
}

// Пиксельный шейдер: визуализируем переданные данные
float4 PS_MatrixTest(VS_OUTPUT input) : COLOR
{
    // === ВАРИАНТЫ ВИЗУАЛИЗАЦИИ ===

    // 1. Визуализация МИРОВЫХ координат как цвета
    float3 color = normalize(input.WorldPos) * 0.5 + 0.5;
    return float4(color, 1.0);

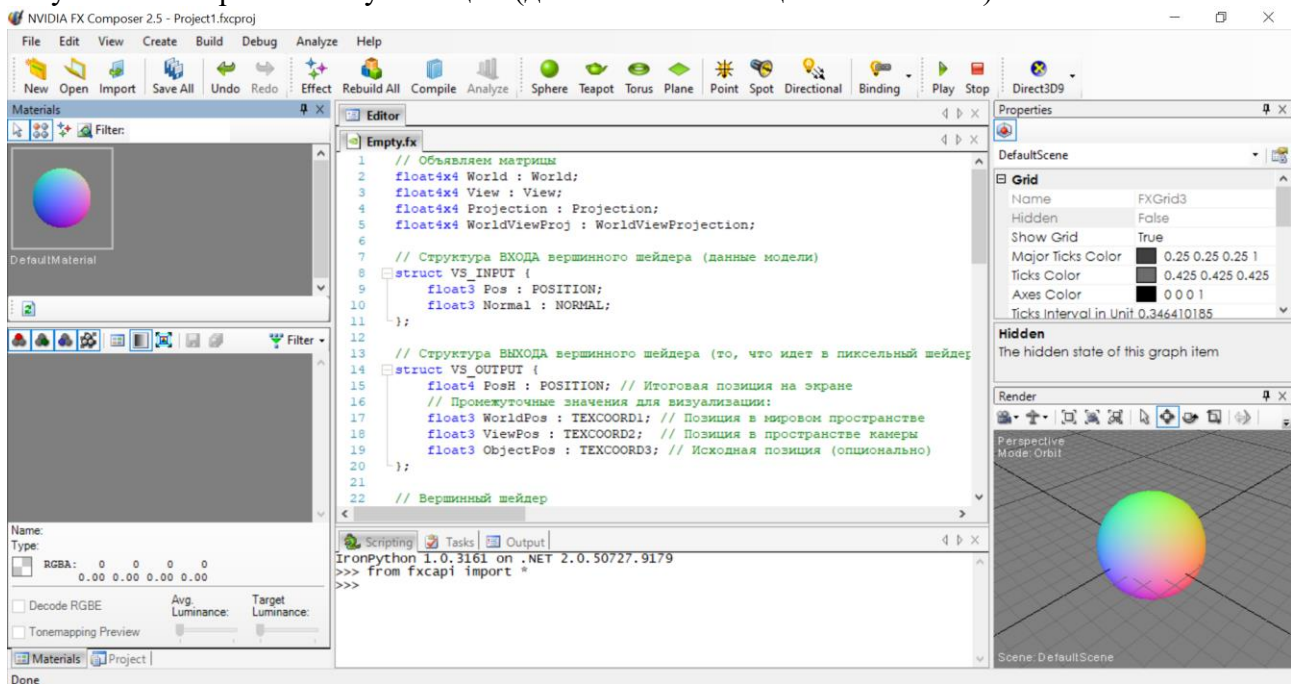
    // 2. Визуализация ВИДОВЫХ координат (пространство камеры)
    // float3 color = normalize(input.ViewPos) * 0.5 + 0.5;
    // return float4(color, 1.0);

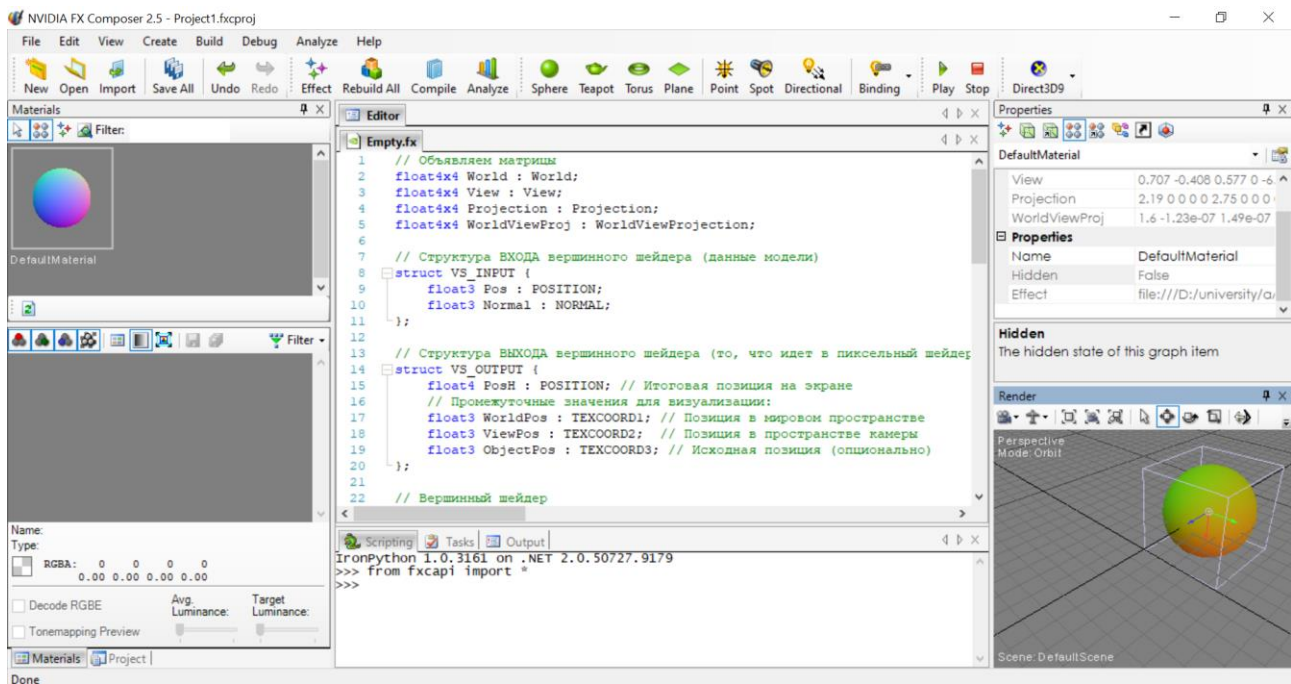
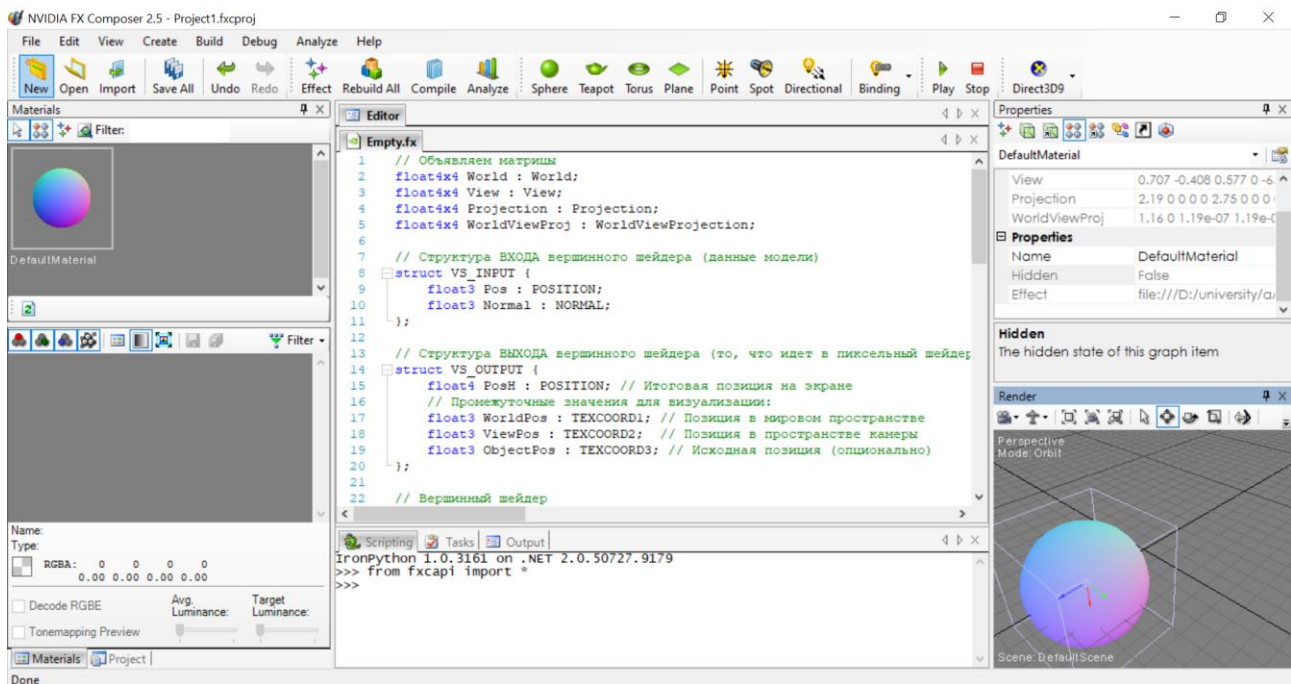
    // 3. Визуализация только ПОЛОЖИТЕЛЬНЫХ мировых координат
    // return float4(input.WorldPos, 1.0);
}

technique technique0
{
    pass p0
    {
        CullMode = None; // Чтобы видеть обе стороны
        VertexShader = compile vs_3_0 VS_MatrixTest();
        PixelShader = compile ps_3_0 PS_MatrixTest();
    }
}

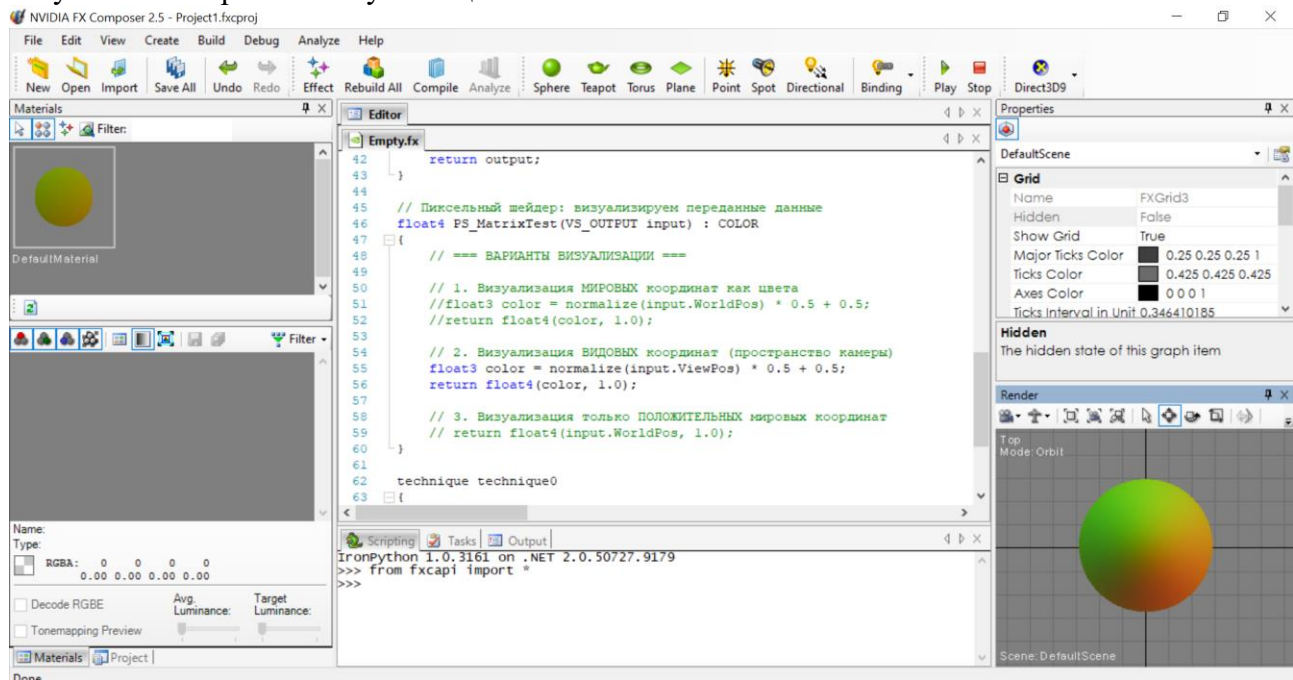
```

Результаты 1 варианта визуализации (двигаем объект и цвета меняются):





Результаты 2 варианта визуализации:



Результаты 3 варианта визуализации:

